2.2 Intervalos

Definición

Un intervalo se define tipicamente como un conjunto continuo de valores entre 2 puntos finales, que pueden ser incluidos (cerrados) o excluidos (abiertos).

- Tipo para definir si el limite es Abierto o Cerrado data Inclusividad = Abierto | Cerrado deriving.

- Tipo de dato para un intervalo.
- 'r' es el tipo de los puntos finales
data Intervalo r = Intervalo (Extremo r) (Extremo r) deriving.

- Tipo de dato para un extremo
data Extremo r = Extremo r Inclusividad deriving.

Tipos

			R. En Haskell
Cerrado	[a,b] N.M	{x∈R | a≤x≤b} D.C. conjunto	data intervalo a = cerrado a a.
Abierto	(a,b) N.M	{x∈R | a≤x<b} D.C. conjunto	data intervalo a = Abierto a a.
semiabierto izq. o semi cerrado dcho.	(a,b] N.M	{x∈R | a<x≤b} D.C. conjunto	data intervalo a = SemiAbierto izquierdo a a.
semiabierto a la derecha o semi cerrado a la izquierda	[a,b) N.M	{x∈R | a≤x<b} D.C. conjunto	data intervalo a = SemiAbierto derecha a a.

FUNCIÓN

Definición

✓ Son funciones que nos ayudan a crear intervalos de una manera más clara y ordenada, esto hace el código mas legible y reutilizable.

✓ En haskell, primero definimos un tipo de dato para representar intervalos

Por ejemplo:

```
data intervalo = Cerrado int int
               | Abierto int int
```

Esto significa que un intervalo puede ser:

```
deriving (show)
```

Tipos

2.4 Operadores

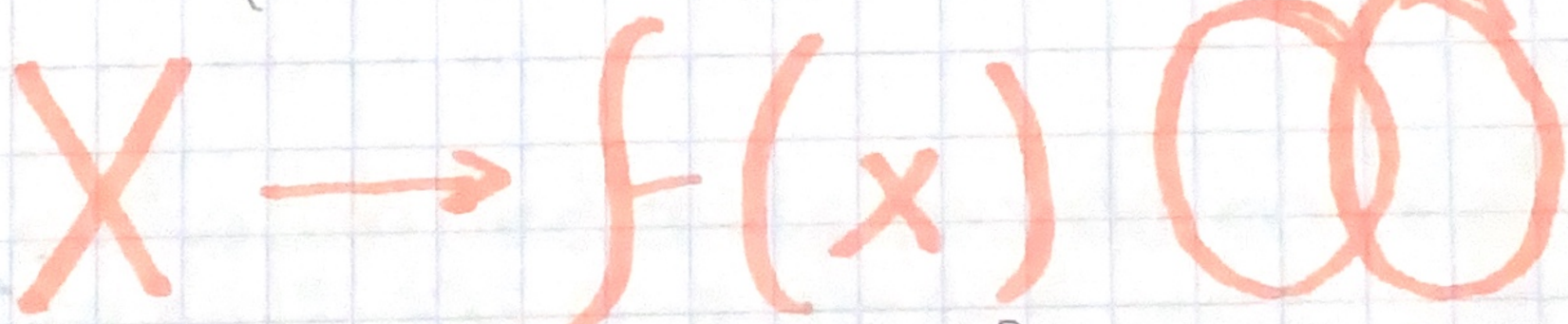
Qué es?

Un operador matemático es definido como una entidad matemática que transforma una función en otra, a menudo representada con símbolos como ∧

Función operador

Math Centre

Una función es una regla que relaciona un número a otro número único.
La entrada a la función es llamada variable independiente, y también llamada el argumento de la función. La salida de la función es variable dependiente.



Los operadores en haskell son funciones A → B que se pueden usar en notación infija. A diferencia de otros lenguajes, haskell permite definir operadores personalizados con precencia y asociatividad especificas.

Declaraciones de Fixity

la fixity controla la precencia y asociatividad de los operadores.

Regla Matemática:

Precencia: 0 ≤ nivel ≤ 9, donde
9 = maxima precedencia

Operadores

Leyes lógicas

- $p \wedge q \equiv q \wedge p$ (conmutatividad)
- $p \vee (q \vee r) \equiv (p \vee q) \vee r$ (asociatividad)

Operadores delistas

$\forall xs, ys, zs: xs ++ (ys ++ zs) =$
$(xs ++ ys) ++ zs$

2.7. Evaluación Perezosa

¿Qué es? la evaluación perezosa (o lazy evaluation) es una estrategia de cálculo en la que haskell retrasa la evaluación de una expresión hasta el momento exacto

Evaluación

→ Para los ejemplos se considera una función

```
mult :: (Int, Int) -> Int
mult (x,y) = x*y
```

→ Evaluación mediante paso de parametros por valor

```
    mult (1+2,2+3)   [Por def de +]
=   mult (3,5)       [ "  "  . De mult]
  = 3*5              [ "  "  . de *]
  = 15
```

→ En la evaluación mediante paso de parámetros por nombres los argumentos pueden evaluarse más veces que en el paso por valor

Se puede usar punteros para compartir valores de expresiones

Evaluación perezosa

```
    cuadrado (1+2)
  = x*x con x=1+2   [Por def. cuadrado]
  = 3*3             [ "  "  . de +
  = 9               [ "  "  . de *]
```